

나:

Project Vault 주제로 3 차 과제를 해야 하는데, 반복문과 메뉴 시스템을 활용해서 여러 프로젝트를 계속 등록하고 조회할 수 있는 구조를 짜줘.

GPT:

좋아. 3 차 과제의 핵심은 기존 2 차에서 만든 프로젝트 분석 로직을 한 번 실행하고 끝내는 것이 아니라, 사용자가 메뉴를 통해 원하는 기능을 계속 선택할 수 있도록 무한 루프 기반 프로그램으로 확장하는 거야.

그래서 전체 구조는 while True 를 중심으로 잡는 것이 가장 적절해. 프로그램을 실행하면 먼저 메뉴를 출력하고, 사용자가 번호를 입력하면 해당 기능이 실행되도록 만들면 된다.

Project Vault 의 경우 3 차에서 가장 자연스러운 메뉴는 다음과 같아.

1. 프로젝트 등록
2. 전체 프로젝트 조회
3. 위험 / 휴면 프로젝트 조회
4. 포트폴리오 완성형 프로젝트 조회
5. 자료 추가 기록
6. 프로그램 종료

이 구조를 쓰면 사용자는 프로젝트를 하나만 입력하고 끝나는 게 아니라, 여러 개를 연속으로 등록하고 필요할 때마다 조건에 맞게 조회할 수 있다.

또한 2 차에서 만든 준비도 점수 계산, 상태 판정, 특별 칭호 부여 로직을 그대로 재사용할 수 있어서 프로그램의 연결성도 좋아진다.

특히 3 차에서는 다음 두 가지가 중요하다.

첫째, 프로젝트 하나는 리스트로 저장하고, 여러 프로젝트는 다시 큰 리스트 `project_vault` 에 차곡차곡 저장해야 한다.

즉, 작은 리스트 하나가 프로젝트 카드 1 개이고, 큰 리스트가 전체 저장소 역할을 하게 된다.

둘째, 전체 프로젝트를 조회하거나 특정 조건의 프로젝트만 찾을 때는 for 문으로 `project_vault` 를 순회해야 한다.

이 과정을 통해 반복문이 실제로 왜 필요한지 보여줄 수 있다.

즉, 3 차의 핵심은

“2 차의 개별 프로젝트 분석 프로그램 → 여러 프로젝트를 반복적으로 관리하는 메뉴형 프로그램”

으로 확장하는 것이다.

나:

반복문 단계에 맞춰 Project Vault 가 실제로 쓸만해 보이게 하려면 어떤 메뉴를 넣는 게 좋고, 각 메뉴에서 어떤 조건문 로직이 있으면 좋은지 정리해줘.

GPT:

좋아. 3 차는 아직 함수/딕셔너리 단계가 아니니까 너무 복잡하게 가기보다, 리스트 + 조건문 + 반복문으로 충분히 구현 가능한 메뉴를 넣는 게 중요하다. 실사용성도 있으면서 과제 범위에도 맞는 메뉴 구성을 추천하면 다음과 같다.

1) 프로젝트 등록

사용자에게 프로젝트명, 분야, 목표, 진행률, 자료 개수, 중요도, 최근 활동 공백일 수, 회고 작성 여부, 링크 개수, 메모 개수 등을 입력받아 하나의 리스트로 저장한다.
이 메뉴는 데이터 입력의 중심이 된다.

2) 전체 프로젝트 조회

for 문을 사용해 전체 리스트를 순회하면서 모든 프로젝트의 정보를 출력한다.
이때 단순 정보만 보여주는 것이 아니라, 각 프로젝트마다 준비도 점수, 상태, 특별 칭호도 함께 출력하면 프로그램 완성도가 높아진다.

3) 위험 / 휴면 프로젝트 조회

조건문을 활용해서 현재 상태가 "위험" 또는 "휴면 프로젝트"인 것만 골라 출력한다.
이 메뉴가 있으면 사용자가 어떤 프로젝트를 먼저 관리해야 하는지 바로 알 수 있어서 프로그램 목적과도 잘 맞는다.

4) 포트폴리오 완성형 프로젝트 조회

특별 칭호가 "포트폴리오 완성형"인 프로젝트만 따로 출력하는 기능이다.

이건 단순 관리 프로그램이 아니라 포트폴리오 저장 기능을 함께 가진 Project Vault 의 개성을 보여준다.

5) 자료 추가 기록

이미 저장된 프로젝트 중 하나를 선택해서 자료 개수, 링크 개수, 메모 개수를 증가시키는 기능을 넣으면 좋다.

이 기능이 있으면 단순 등록 프로그램이 아니라, 저장된 프로젝트를 계속 업데이트할 수 있는 관리 시스템처럼 보인다.

0) 종료

사용자가 원할 때 프로그램을 종료할 수 있도록 break 를 사용한다.

이런 메뉴 구성이 좋은 이유는,

단순히 반복문을 썼다는 수준이 아니라 실제로 반복해서 쓰는 프로그램 구조가 만들어지기 때문이다.

과제 관점에서도 반복문 사용 목적이 매우 명확해지고, 시나리오와도 자연스럽게 연결된다.

나:

그럼 조회 메뉴마다 2 차 때 만든 상태 판정과 특별 칭호 로직을 그대로 재사용해도 괜찮아?

GPT:

오히려 그게 가장 좋다.

왜냐하면 3 차는 완전히 새로운 계산식을 만드는 단계가 아니라, 기존 로직을 여러 데이터에 반복 적용하는 단계이기 때문이다.

예를 들어 전체 조회 메뉴에서는 각 프로젝트마다

- 준비도 점수 계산
- 휴면/순항/진행 중/정체/위험 상태 판정
- 포트폴리오 완성형 / 아카이브 마스터 / 폭풍 성장 프로젝트 / 자료만 많은 프로젝트 같은 칭호 부여

를 순서대로 다시 실행하면 된다.

즉, 2 차가 "프로젝트 1 개를 분석하는 단계"였다면,

3 차는 "그 분석을 여러 프로젝트에 반복 적용하는 단계"라고 보면 된다.

이렇게 하면 프로그램이 자연스럽게 발전한 것처럼 보여서 과제 흐름도 훨씬 좋다.

나:

프로젝트를 여러 개 저장했는데도 출력해보면 첫 번째 프로젝트만 계속 보이는 문제가 생길 수 있을 것 같은데, 이런 건 왜 생기고 어떻게 고치면 돼?

GPT:

이건 3 차에서 자주 나오는 전형적인 문제다.

원인은 프로젝트를 여러 개 리스트에 저장해 두었더라도, 출력할 때 `project_vault[0]`처럼 특정 인덱스만 직접 참조하면 첫 번째 프로젝트만 반복해서 보이게 되기 때문이다.

예를 들어 사용자가 프로젝트를 3 개 등록했더라도, 코드가 계속 `[0]`만 보고 있으면 첫 번째 프로젝트 정보만 출력된다.

이건 데이터를 저장하는 것과 데이터를 "순회해서 처리하는 것"을 구분하지 못했을 때 생기는 문제다.

이 문제를 해결하려면 전체 리스트를 `for` 문으로 순회해야 한다.

예를 들면

```
for project in project_vault:  
    print(project)
```

이런 구조로 바꾸면 저장된 모든 프로젝트를 하나씩 꺼내 볼 수 있다.

즉,

- 잘못된 방식: `project_vault[0]`, `project_vault[1]`처럼 직접 하나씩 지정
- 올바른 방식: `for project in project_vault:`로 전체를 순회

이렇게 바꾸면 프로젝트 수가 늘어나도 코드를 다시 고칠 필요 없이 자동으로 전부 출력된다.

3 차 과제에서 반복문을 쓰는 가장 대표적인 이유가 바로 이 부분이다.

나:

메뉴 시스템 만들 때 사용자가 이상한 값을 입력하면 프로그램이 꼬일 수도 있잖아. 이걸 어떻게 처리하면 좋아?

GPT:

맞아. 메뉴 시스템에서 가장 기본적인지만 중요한 부분이 바로 잘못된 입력 처리다.

예를 들어 메뉴 번호를 1, 2, 3, 4, 5, 0 중 하나로 입력해야 하는데 사용자가 9, abc, 빈 문자열 같은 걸 넣으면 프로그램 흐름이 어색해질 수 있다.

이 문제는 보통 다음 두 가지 이유로 생긴다.

첫째, 메뉴 입력값을 받을 때 “정상 입력만 들어올 것”이라고 가정하는 경우

둘째, if-elif 조건에 없는 경우를 처리하는 마지막 else 가 없는 경우

가장 쉬운 해결 방법은 메뉴 값을 문자열로 받고, 각 메뉴 번호를 문자열 "1", "2"처럼 비교한 뒤, 마지막에 else 로 예외 상황을 처리하는 것이다.

예를 들어

```
menu = input("메뉴 번호를 선택하세요: ")
```

```
if menu == "1":
```

```
...
```

```
elif menu == "2":
```

```
...
```

```
elif menu == "0":
```

```
...
```

```
else:
```

```
    print("올바른 메뉴 번호를 입력하세요.")
```

이렇게 만들면 잘못된 입력이 들어와도 프로그램이 죽지 않고 다시 메뉴로 돌아간다.

그리고 3 차는 무한 루프 기반이기 때문에, 잘못된 입력이 나와도 프로그램이 종료되면 안 되고 계속 다시 선택할 수 있게 만드는 것이 중요하다.

그래서 while True 안에서 메뉴를 반복 출력하는 구조와 else 안내문이 같이 있어야 안정적인 프로그램이 된다.

나:

그럼 숫자인지까지 검사해야 해?

GPT:

꼭 숫자 판별까지 하지 않아도 된다.

왜냐하면 메뉴 번호 자체를 문자열 "1", "2"로 비교하면, 사용자가 숫자가 아닌 문자를 입력해도 어차피 어떤 조건에도 맞지 않아서 마지막 else 로 처리되기 때문이다.

즉 3 차 수준에서는

“메뉴 입력을 문자열로 받고, 허용된 번호가 아니면 다시 입력하게 한다”

정도만 해도 충분히 안정적이고 과제 범위에도 잘 맞는다.

나:

조건문이 길어질수록 상태 판정 기준이 헷갈리는데, 준비도 점수랑 공백일 수랑 특별 칭호 조건이 같이 있으면 순서를 어떻게 잡아야 해?

GPT:

이 문제도 아주 중요하다.

조건문은 단순히 조건만 맞으면 되는 게 아니라, 어떤 조건을 먼저 검사하느냐에 따라 결과가 완전히 달라질 수 있다.

Project Vault에서는 대표적으로 이런 문제가 생긴다.

예를 들어 어떤 프로젝트가

- 준비도 점수는 높다
- 하지만 최근 활동 공백일 수가 10 일이다

이 경우 사람이 보기에는 “오래 방치된 프로젝트”니까 휴면으로 보는 게 맞다.

그런데 만약 코드에서 준비도 점수 조건을 먼저 검사하면,

```
if readiness_score >= 90:
```

```
    status = "순항"
```

```
elif inactive_days >= 7:  
    status = "휴면 프로젝트"
```

이렇게 되어 있을 때, 공백일 수가 10 일이어도 준비도 점수가 높다는 이유로 먼저 "순항"이 나와버린다.

즉, 논리적으로는 휴면이 맞는데 코드상으로는 순항이 되는 오류가 발생한다.

그래서 상태 판정에서는 항상 우선순위가 높은 조건을 먼저 뒤야 한다.

Project Vault 의 경우에는 다음 순서가 가장 자연스럽다.

1. 최근 활동 공백일 수가 7 일 이상이면 → 무조건 "휴면 프로젝트"
2. 그 외에는 준비도 점수 기준으로
 - 90 이상 → "순항"
 - 70 이상 → "진행 중"
 - 50 이상 → "정체"
 - 그 외 → "위험"

즉 휴면 판정은 준비도 점수보다 우선해야 한다.

특별 칭호도 마찬가지로.

가장 강한 조건부터 위에 두는 것이 좋다. 예를 들어

1. 포트폴리오 완성형
2. 아카이브 마스터
3. 폭풍 성장 프로젝트
4. 자료만 많은 프로젝트
5. 일반 프로젝트

이런 순서로 두면, 한 프로젝트가 여러 조건에 동시에 걸리더라도 가장 대표성이 높은 칭호가 먼저 부여된다.